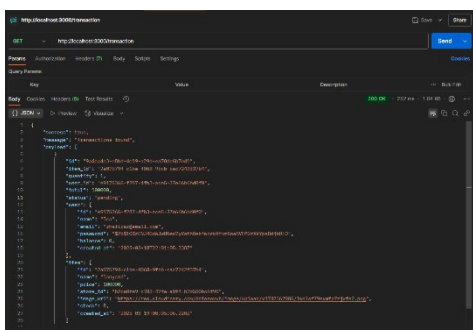
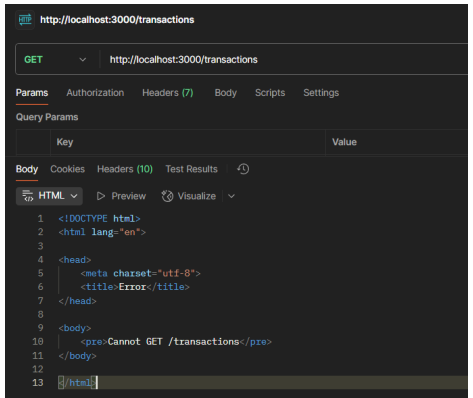


## PRAKTIKUM SISTEM BASIS DATA

|             |                               |                  |                       |
|-------------|-------------------------------|------------------|-----------------------|
| <b>Nama</b> | <b>Zhafira Zahra Alfarisy</b> | <b>No. Modul</b> | <b>6</b>              |
| <b>NPM</b>  | <b>2306250636</b>             | <b>Tipe</b>      | <b>Tugas Tambahan</b> |

1. Melanjutkan backend yang sudah dibuat
2. Get response sesuai dokumentasi

| Method | Endpoint     | Success                                                                            | Failed                                                                               |
|--------|--------------|------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| GET    | /transaction |  |  |

3. Meningkatkan kode express pada bagian scalability, security, dan error handling.
4. Hal-hal yang telah anda lakukan untuk scalability, security, dan error handling selama modul 5 dan 6

| Type                  | Implementasi                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Deskripsi                                                                                                                                                       |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Error Handling</b> | <p><b>Pada Controllers</b></p> <ul style="list-style-type: none"> <li>• Pengecekan Input</li> </ul> <pre> exports.createStore = async (req, res) =&gt; {   if (!req.body.name    !req.body.address) {     return baseResponse(res, false, 400, "Name and address are required");   }    try {     const store = await storeRepository.createStore(req.body);     return baseResponse(res, true, 201, "Store created", store);   } catch (error) {     return baseResponse(res, false, 500, "Error creating store", error);   } }; </pre> | <p>Pengecekan dilakukan sebelum membuat store, dikarenakan kemungkinan terjadinya kesalahan input dari client yang tidak menyertakan nama atau alamat toko,</p> |

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                           |                           |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                                                                                                                                                                           | yang merupakan data wajib |
| <ul style="list-style-type: none"><li>Try-Catch</li></ul> <pre>exports.getAllStores = async (req, res) =&gt; {   try {     const stores = await storeRepository.getAllStores();     return baseResponse(       res,       true,       200,       "Stores retrieved successfully",       stores     );   } catch (error) {     return baseResponse(res, false, 500, "Error retrieving stores", error);   } };</pre>                                                                                                             | Error handling dilakukan untuk mengantisipasi kemungkinan kegagalan saat mengambil data dari repository, seperti koneksi database yang gagal atau error internal lainnya. |                           |
| <ul style="list-style-type: none"><li>Pengecekan email dan password saat registration</li></ul> <pre>exports.registerUser = async (req, res) =&gt; {   const { email, password, name } = req.body;    if (!isValidEmail.test(email)) {     return baseResponse(res, false, 400, "Invalid email format", null);   }    if (!isValidPassword.test(password)) {     return baseResponse(res, false, 400, "Password must be at least 8 characters long and contain at least 1 letter, 1 number, and 1 special char");   } };</pre> | Pengecekan dilakukan sebelum mendaftarkan user, dikarenakan kemungkinan terjadinya kesalahan validasi format email dan password yang tidak sesuai standar keamanan.       |                           |
| <ul style="list-style-type: none"><li>Pengecekan ketersediaan user dan item saat transaksi</li></ul> <pre>    } catch (error) {       if (error.message === "User not found") {         return baseResponse(res, false, 404, "User not found", null);       }       if (error.message === "Item not found") {         return baseResponse(res, false, 404, "Item not found", null);       }       baseResponse(res, false, 500, error.message    "Server Error", null);     }   }; };</pre>                                    | Penanganan error berdasarkan pesan exception, dikarenakan kemungkinan user atau item tidak ditemukan dalam database saat proses pembuatan transaksi.                      |                           |

|             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |                                                                                                                                                                                                                        |
|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|             | <ul style="list-style-type: none"> <li>Validasi saldo saat pembayaran</li> </ul> <pre> exports.payTransaction = async (req, res) =&gt; {   const transactionId = req.params.id;    try {     const transaction = await transactionRepository.payTransaction(transactionId);     baseResponse(res, true, 200, "Payment successful", transaction);   } catch (error) {     if (error.message === "Transaction not found") {       return baseResponse(res, false, 404, "Transaction not found", null);     }     if (error.message === "User not found") {       return baseResponse(res, false, 404, "User not found", null);     }     if (error.message === "Not enough balance") {       return baseResponse(res, false, 400, "Not enough balance", null);     }     baseResponse(res, false, 500, "Failed to pay", null); // Updated error message   } }; </pre> | Penanganan error dilakukan untuk mengantisipasi kemungkinan saldo pengguna tidak mencukupi untuk melakukan pembayaran transaksi.                                                                                       |
|             | <ul style="list-style-type: none"> <li>Validasi store_id sebelum membuat item</li> </ul> <pre> const storeExists = await itemRepository.checkStoreExists(store_id); if (!storeExists) {   return baseResponse(res, false, 404, "Store doesn't exist", null); } </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | Pengecekan dilakukan sebelum membuat item, dikarenakan kemungkinan store_id yang diberikan tidak merujuk pada toko yang ada, sehingga mencegah item orphan.                                                            |
| Scalability | <p>Pada pg.database.js</p> <ul style="list-style-type: none"> <li>pg.Pool untuk koneksi database</li> </ul> <pre> const { Pool } = require("pg");  const pool = new Pool({   connectionString: process.env.PG_CONNECTION_STRING,   ssl: {     rejectUnauthorized: false,   }, }); </pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Implementasi menggunakan pg.Pool karena sistem bisa menerima banyak request secara bersamaan. Pooling memungkinkan reuse koneksi database tanpa buka koneksi baru tiap kali, sehingga lebih scalable saat trafik naik. |

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|          | <ul style="list-style-type: none"> <li>Abstraksi query sebagai fungsi terpisah</li> </ul> <pre>const query = async (text, params) =&gt; {   try {     const res = await pool.query(text, params);     return res;   } catch (error) {     console.error("Error executing query", error);   } };</pre>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <p>Logika query dipisah jadi fungsi sendiri karena aplikasi bisa berkembang dengan banyak modul. Pendekatan ini bikin akses database lebih terpusat dan gampang di-maintain kalau ada perubahan sistem.</p>                                                                                                                                                                                                                                                             |
| Security | <p>Pada user.controller.js</p> <pre>exports.registerUser = async (req, res) =&gt; {   const { email, password, name } = req.body;    if (!regEmail.test(email)) {     return baseResponse(res, false, 400, "Invalid email format", null);   }    if (!regPassword.test(password)) {     return baseResponse(res, false, 400, "Password must be at least 8 characters long and contain at least 1 letter, 1 number, and 1 special character", null);   }    const hashedPassword = await bcrypt.hash(password, 10);    exports.login = async (req, res) =&gt; {     const { email, password } = req.body;      if (!email    !password) {       return baseResponse(res, false, 400, "Email and password are required", null);     }      try {       const user = await userRepository.getUserByEmail(email);        if (!user) {         return baseResponse(res, false, 400, "Invalid email or password", null);       }        const isPasswordValid = await bcrypt.compare(password, user.password);        if (!isPasswordValid) {         return baseResponse(res, false, 400, "Invalid email or password", null);       }        return baseResponse(res, true, 200, "Login success", user);     } catch (error) {}     return baseResponse(res, false, 500, "Error logging in", error);   } };</pre> | <ul style="list-style-type: none"> <li>Validasi dilakukan untuk mencegah input yang tidak aman, seperti email invalid atau password lemah. Ini mencegah potensi abuse dan brute-force terhadap sistem auth.</li> <li>Password user tidak disimpan dalam bentuk asli, tapi di-hash pakai bcrypt. Ini penting untuk melindungi data sensitif jika database bocor.</li> <li>Password yang dikirim user dibandingkan dengan hash di database menggunakan bcrypt,</li> </ul> |

|  |                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                             |
|--|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  |                                                                                                                                                                                              | <p>bukan dicek langsung. Ini menjaga agar password tetap aman dalam proses autentikasi.</p> <ul style="list-style-type: none"> <li>Error message dibuat umum agar attacker tidak bisa tahu apakah email yang digunakan valid atau tidak. Ini mengurangi resiko user enumeration.</li> </ul> |
|  | <p>Pada index.js</p> <ul style="list-style-type: none"> <li>CORS</li> </ul> <pre>app.use(   cors({     origin: "https://os.netlabdte.com",     methods: "GET,POST,PUT,DELETE",   }) );</pre> | <p>CORS dikonfigurasi untuk hanya mengizinkan request dari domain tertentu, dikarenakan kemungkinan request bisa datang dari web yang tidak sah. Ini mencegah akses API dari sumber asing</p>                                                                                               |